FASTROUTE KAYSERI

Urban Logistics & Distribution System

COMP206

Semester Project Report

Eren Bülbül

Tuğçe Taşcı

Zeynep Aybuke Ilık

Ozan Akdeniz

Hamza Mete Erbölükbaş

1. Company Name

FastRoute Kayseri

2. Mission Statement

FastRoute Kayseri aims to improve package delivery and warehouse management operations in Kayseri by using efficient data structures and graph algorithms. The system organizes package records, manages incoming packages, prepares deliveries, supports truck loading, stores address information, and optimizes city routes. The main mission of the company is to provide a fast, reliable, and organized logistics service by selecting the most suitable data structure for each operational task.

3. System Overview

This Java console application simulates an urban logistics and distribution system. The project uses Object-Oriented Programming principles by separating the system into different classes such as Package, SinglyLinkedList, DoublyLinkedList, PackageQueue, PackageStack, AVLTree, Edge, Graph, and Main. Each class has a specific responsibility, which makes the code easier to understand, test, and maintain.

System Part	Structure Used	Purpose
Master Registry	Singly Linked List	Stores the daily record of packages entering the warehouse.
Intake Buffer	Doubly Linked List	Manages packages arriving at the warehouse and removes packages for processing.
Standard Delivery	Queue	Processes packages with FIFO logic.
Truck Loading	Stack	Loads and unloads packages with LIFO logic.
Address Directory	AVL Tree	Stores neighborhoods and customer IDs with balanced search performance.
City Routing	Weighted Graph	Represents neighborhoods and roads for routing and network optimization.
File Loading	BufferedReader/FileReader	Loads package and map information from packageData.txt and mapData.txt.

4. Explanation of Operations

Singly Linked List: addRecord() appends package records to the master registry. displayLog() prints the daily package log.

Doubly Linked List: insertAtTail() adds a newly arrived package to the end of the intake buffer. removeFromHead() moves the first package out of the buffer.

Queue: enqueue() adds a package to the rear of the delivery line. dequeue() removes the front package first, following FIFO order.

Stack: push() loads a package onto the truck. pop() unloads the most recently loaded package first, following LIFO order.

AVL Tree: insert() stores neighborhood and customer information. search() finds address records efficiently because the tree remains balanced.

Graph: addEdge() creates road connections. Dijkstra calculates the shortest delivery route. Prim MST finds an efficient network connecting all neighborhoods.

5. Complexity Analysis Table

The table below summarizes the time complexity of the main operations used in the system.

Component	Operation	Time Complexity	Explanation
Singly Linked List	addRecord(Package pkg)	O(n)	Traverses the list to append a package at the end.
Singly Linked List	displayLog()	O(n)	Visits every package record once.
Singly Linked List	isEmpty()	O(1)	Checks only the head pointer.
Doubly Linked List	insertAtTail(Package pkg)	O(1)	Uses the tail pointer to insert directly at the end.
Doubly Linked List	removeFromHead()	O(1)	Updates the head pointer directly.
Doubly Linked List	displayBuffer()	O(n)	Traverses all buffer nodes once.
Queue	enqueue(Package pkg)	O(1)	Uses the rear pointer for direct insertion.
Queue	dequeue()	O(1)	Uses the front pointer for direct removal.
Queue	displayQueue()	O(n)	Visits every queue node once.
Stack	push(Package pkg)	O(1)	Inserts directly at the top.
Stack	pop()	O(1)	Removes directly from the top.
Stack	displayStack()	O(n)	Visits every stack node once.

AVL Tree	insert(String neighborhood, String customerID)	O(log n)	Tree height remains balanced after rotations.
AVL Tree	search(String neighborhood)	O(log n)	Balanced tree reduces search height.
AVL Tree	displayInOrder()	O(n)	Visits every tree node once.
Graph	addEdge(String source, String destination, int weight)	O(V)	May search vertex names before adding the edge.
Graph	displayGraph()	O(E)	Displays every stored edge once.
Graph	calculateShortestPath(start, end) - Dijkstra	O(V^2)	Uses simple loops to select the next closest vertex.
Graph	calculateMST() - Prim	O(V^2)	Repeatedly selects the minimum connecting edge using loops.
File Loading	Read mapData.txt	O(m)	Reads each map line once.
File Loading	Read packageData.txt	O(p)	Reads each package line once.

6. Conclusion

FastRoute Kayseri successfully demonstrates how different data structures can be applied to real logistics problems. The project uses linked lists for package records and intake buffering, a queue for delivery order, a stack for truck loading, an AVL tree for address lookup, and graph algorithms for route optimization. The system also supports external data loading from text files, which makes the simulation more realistic and flexible.